Enterprise Data Governance

> **Document Owner:** Chief Data Officer | **Status:** Approved | **Last Updated:** February 2026

Overview

This document establishes the enterprise data governance framework for our Databricks platform built on Unity Catalog. It defines the policies that ensure data quality, security, compliance, and business value across all data and AI assets.

Golden Rules

ID	Rule	Severity	WAF Pillar
EA-01	All data assets must have business context documentation	Critical	Governance
EA-02	PII columns must be tagged and classified	Critical	Governance
EA-03	Every data domain must have an assigned data steward	Required	Governance
EA-04	All compute usage must be attributed with tags	Critical	Governance
EA-05	Required custom tags: team, cost_center, environment	Required	Governance
EA-06	All serverless workloads must use assigned budget policies	Critical	Security
EA-07	All new projects require architecture review	Required	Governance
EA-08	Rule exceptions require documented approval	Required	Governance
EA-09	Use Governed Tags for UC object metadata (showback/chargeback)	Critical	Governance
EA-10	Treat data as a product with defined SLAs and ownership	Critical	OpEx
EA-11	Define data contracts for cross-domain interfaces	Required	Governance
EA-12	Assess and track AI readiness across data domains	Required	Governance

---

EA-01: Data Documentation

Rule

All tables must have LLM-friendly COMMENT documentation at both table and column level.

Why It Matters

- Enables discoverability and understanding across teams
- Powers natural language queries via Genie
- Supports regulatory compliance documentation

Implementation

```sql
-- Table-level comment (dual-purpose for business users and LLMs)
COMMENT ON TABLE gold.dim_customer IS
'Gold layer customer dimension with current and historical records.
Business: Primary customer reference for all analytics.
Technical: SCD Type 2 with is_current flag.';

-- Column-level comment
COMMENT ON COLUMN gold.dim_customer.lifetime_value IS
'Customer lifetime value in USD. Business: Total revenue attributed
to this customer. Technical: Calculated daily, includes returns.';
```

EA-02: PII Classification

Rule

All columns containing personally identifiable information must be tagged with `pii = true` and the appropriate `pii_type`.

Why It Matters

- Required for GDPR, CCPA, and HIPAA compliance
- Enables automated data protection policies
- Supports audit and access control requirements

Implementation

```sql
-- Tag PII columns
ALTER TABLE gold.dim_customer
ALTER COLUMN email SET TAGS ('pii' = 'true', 'pii_type' = 'email');

ALTER TABLE gold.dim_customer
ALTER COLUMN phone SET TAGS ('pii' = 'true', 'pii_type' = 'phone');

-- Tag table classification
ALTER TABLE gold.dim_customer
SET TAGS ('data_classification' = 'confidential');
```

Classification Levels

Level	Description	Examples
Confidential	Highly sensitive, PII, financial	SSN, credit card, salary
Internal	Business sensitive, not public	Sales figures, strategies
Public	Safe for external sharing	Product specs, marketing

EA-03: Data Stewardship

Rule

Every data domain must have an assigned data steward responsible for governance within that domain.

Why It Matters

- Clear accountability for data quality and access decisions
- Reduces bottlenecks in access request processing
- Enables domain expertise in data definitions

Steward Responsibilities

Responsibility	Description
Define	Business definitions and quality rules
Approve	Access requests and schema changes
Monitor	Quality metrics and usage patterns
Resolve	Data issues and conflicting definitions

EA-04: Usage Attribution Tags

Rule

All compute usage must be attributed with tags for cost tracking, reporting, and budgeting.

Why It Matters

- Enables accurate cost allocation (showback/chargeback)
- Supports budget enforcement and anomaly detection
- Required for capacity planning

Default Tags (Automatic)

Tag	Value
`Vendor`	`Databricks`
`ClusterId`	Internal cluster ID

Tag	Value
`ClusterName`	Cluster name
`Creator`	User email
`RunName`	Job name (jobs only)

EA-05: Required Custom Tags

Rule

All compute resources must include these custom tags:

Tag	Required	Example
`team`	Yes	`data-engineering`
`cost_center`	Yes	`CC-1234`
`environment`	Yes	`dev`, `staging`, `prod`
`project`	Recommended	`customer-360`
`workload_type`	Recommended	`etl`, `ml-training`, `analytics`

Implementation

Cluster Policy Enforcement:

```
{
  "custom_tags.team": { "type": "required" },
  "custom_tags.cost_center": { "type": "required" },
  "custom_tags.environment": {
    "type": "allowlist",
    "values": ["dev", "staging", "prod"]
  }
}
```

EA-06: Serverless Budget Policies

Rule

All serverless workloads (notebooks, jobs, pipelines, model serving) must use assigned budget policies for cost attribution.

Why It Matters

- Enables cost tracking for serverless compute (which doesn't have cluster tags)
- Tags automatically appear in `system.billing.usage`

- Supports team-level budget enforcement

Setup Steps

1. Navigate to **Settings** → **Compute** → **Serverless budget policies**
2. Create policy with required tags
3. Assign User or Manager permission to appropriate groups
4. Users select policy when creating serverless resources

Query Tagged Usage

```sql
SELECT
    usage_date,
    custom_tags:team AS team,
    custom_tags:cost_center AS cost_center,
    SUM(list_cost) AS total_cost
FROM system.billing.usage
WHERE sku_name LIKE '%SERVERLESS%'
GROUP BY 1, 2, 3;
```

EA-07: Architecture Review

Rule

All new data platform projects must complete architecture review before implementation begins.

Why It Matters

- Ensures alignment with platform standards
- Identifies potential issues early
- Reduces rework and technical debt

Review Checklist

- [ ] Medallion architecture design documented
- [ ] Gold layer star schema (ERD) reviewed
- [ ] PII data identified and classification assigned
- [ ] Data quality rules defined
- [ ] Security requirements documented
- [ ] Cost estimates provided

EA-08: Exception Approval

Rule

Any deviation from Golden Rules requires documented approval with business justification, risk assessment, and remediation timeline.

Why It Matters

- Maintains governance integrity
- Creates audit trail for compliance
- Ensures exceptions are temporary, not permanent

Exception Request Requirements

Field	Description
Rule being bypassed	Which Golden Rule
Business justification	Why exception is necessary
Risk assessment	Impact if exception granted
Remediation plan	How/when compliance will be achieved
Approver	Platform Architect or Architecture Board

EA-09: Governed Tags for UC Objects

Rule

Use Governed Tags (account-level tags with enforced allowed values) for consistent metadata on Unity Catalog objects to support showback/chargeback.

Why It Matters

- Enforces consistent tagging across all workspaces
- Supports attribute-based access control (ABAC)
- Enables accurate cost allocation to data assets

Required Governed Tags

Tag	Allowed Values	Apply To
`cost_center`	Organization-specific codes	Catalogs
`business_unit`	Sales, Engineering, etc.	Catalogs
`data_owner`	Email or team name	Schemas
`data_classification`	public, internal, confidential, restricted	Tables

Governed vs User-Defined Tags

Feature	Governed	User-Defined
Policy enforcement	✅	❌
Permission control	✅ ASSIGN permission	Any user with MANAGE

Feature	Governed	User-Defined
Predefined values	✅	Any value
Inheritance to children	✅	❌

Implementation

```
-- Apply governed tag to catalog
ALTER CATALOG sales_data SET TAGS ('cost_center' = 'CC-1234');

-- Apply governed tag to schema
ALTER SCHEMA sales_data.gold SET TAGS ('data_classification' =
'confidential');
```

EA-10: Data as a Product

Rule

Each data domain must publish its Gold layer assets as data products with defined ownership, SLAs, quality guarantees, and discoverability metadata.

Why It Matters

- Shifts data from a byproduct of processes to a first-class organizational asset
- Enables self-service consumption without tribal knowledge
- Creates accountability for data quality at the source
- Reduces duplicative pipelines across business units

Data Product Requirements

Attribute	Description	Example
Owner	Team or individual accountable	`analytics-engineers` group
SLA	Freshness and availability commitment	Data refreshed by 6 AM UTC daily
Quality Score	Measurable quality threshold	>99% completeness, >95% accuracy
Schema Contract	Published and versioned schema	YAML schema in Git
Documentation	Business context and lineage	Table/column COMMENTs + ERD
Discoverability	Searchable in Unity Catalog	Tags, comments, governed tags

Implementation

```
-- Tag a Gold table as a published data product
ALTER TABLE gold.dim_customer SET TAGS (
```

```
    'data_product' = 'true',
    'data_product_owner' = 'customer-analytics',
    'data_product_sla' = 'daily-6am-utc',
    'data_product_version' = 'v2.1'
);
```

Anti-Patterns

- Publishing raw Bronze tables as data products (insufficient quality)
- Data products without documented SLAs (unmeasurable commitments)
- Individual ownership instead of team ownership (single point of failure)

EA-11: Data Contracts

Rule

Cross-domain data interfaces must define explicit data contracts specifying schema, quality rules, freshness guarantees, and breaking change policies.

Why It Matters

- Prevents downstream breakage from upstream schema changes
- Establishes clear expectations between producers and consumers
- Enables independent development velocity for domain teams
- Creates an auditable agreement for data exchange

Contract Components

Component	Description
Schema	Column names, types, nullability
Quality Rules	DLT expectations or DQX checks that must pass
Freshness	Maximum acceptable data staleness
Volume	Expected row count range per refresh
Breaking Change Policy	How schema changes are communicated and versioned
Deprecation Policy	Minimum notice period before removing columns/tables

Implementation

Data contracts are enforced through a combination of:

1. **YAML schema files** checked into Git (schema contract)
2. **DLT Expectations** (quality contract)
3. **Lakehouse Monitoring alerts** (freshness and volume contract)
4. **Table COMMENTs** referencing contract version

```yaml
# Example: data_contract_dim_customer.yaml
contract:
  name: dim_customer
  version: "2.1"
  owner: customer-analytics
  schema: gold
  freshness_sla: "daily by 06:00 UTC"
  breaking_change_policy: "30-day deprecation notice via #data-contracts
channel"
  quality_rules:
    - "customer_key IS NOT NULL"
    - "customer_id IS NOT NULL"
    - "is_current IN (true, false)"
  volume_expectation:
    min_rows: 100000
    max_rows: 50000000
```

EA-12: AI Readiness Assessment

Rule

Each data domain must assess and track AI readiness, ensuring Gold layer data products are structured, documented, and quality-controlled for ML feature extraction and GenAI grounding.

Why It Matters

- Prevents rework when ML/AI teams discover data is unusable for training
- Ensures data investments support both BI and AI workloads
- Identifies gaps in data quality, completeness, and labeling early
- Aligns data engineering with emerging AI use cases

AI Readiness Dimensions

Dimension	Assessment Criteria	Target
Completeness	% of columns with <5% null rates	>90% of key columns
Freshness	Data updated within SLA	100% of scheduled refreshes
Documentation	LLM-friendly comments on all tables/columns	100% coverage
Constraints	PK/FK defined for join discovery	All Gold tables
Feature Readiness	Numeric/categorical columns clean of NaN/Inf	All potential feature columns
Lineage	Full lineage from source to Gold	All Gold tables
Volume	Sufficient history for training (typically 12+ months)	Domain-dependent

Implementation

Track readiness per domain using a governance table:

```
CREATE TABLE monitoring.ai_readiness_scores (
    domain STRING,
    assessment_date DATE,
    completeness_score DECIMAL(5,2),
    documentation_score DECIMAL(5,2),
    constraint_score DECIMAL(5,2),
    feature_readiness_score DECIMAL(5,2),
    overall_score DECIMAL(5,2),
    assessor STRING
);
```

Governance Model

Our organization uses a **federated governance model** with centralized standards and domain-level execution:

Aspect	Central Platform Team	Domain Teams
Owner	Platform Team	Data Stewards + Domain Engineers
Scope	Metastore, global policies, security baselines	Catalog/domain data, quality rules, access
Standards	Naming conventions, tagging, security, compute	Data modeling, quality thresholds, SLAs
Enforcement	Cluster policies, governed tags, UC permissions	DLT expectations, data contracts, monitors
Examples	Security baselines, naming standards, budget policies	Access approvals, quality rules, domain SLAs

Federated Governance Principles

Principle	Description
Centralize what must be consistent	Security, identity, naming, networking
Federate what requires domain knowledge	Data quality rules, access decisions, modeling choices
Automate enforcement	Cluster policies, governed tags, DLT expectations
Measure compliance	System tables, audit logs, governance dashboards

---

Governance Metrics

Metric	Target	Alert Threshold
Documentation coverage	100%	<90%
Classification coverage	100%	<90%
Steward assignment	100%	<100%
Quality rule coverage	>90%	<80%
Tag compliance	100%	<95%

Related Documents

- Roles & Responsibilities
- Implementation Workflow
- Data Modeling
- Naming & Comment Standards
- Tagging Standards
- Data Quality Standards
- Unity Catalog Standards
- Architecture Evolution & Decision Heuristics

References

- Best Practices for Data and AI Governance
- Unity Catalog Governance
- Usage Attribution Tags
- Serverless Budget Policies
- Governed Tags